⚙️ Practical Tasks Before Component 1 Completion (Ubuntu + eBPF + Cilium)

Component Focus: Intent-Aware Intelligent Traffic Routing
Owner: Samarasinghe P.P. – IT22036384
Environment: Ubuntu Linux (Kernel ≥ 5.x), C, eBPF, Cilium

🧰 1. Environment Setup (Ubuntu Machine)

🔧 Install eBPF Toolchain

```
sudo apt update
sudo apt install -y clang llvm libelf-dev gcc make iproute2 iputils-ping
libbpf-dev linux-headers-$(uname -r) bpftool
```

🧪 Install `bcc` for debugging and testing

```
sudo apt install -y bpfcc-tools python3-bcc
```

🐳 Install and Set Up Cilium (for Minikube)

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-
linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

minikube start --network-plugin=cni --cni=cilium
```

Install Cilium CLI:

```
curl -L --remote-name https://github.com/cilium/cilium-
cli/releases/latest/download/cilium-linux-amd64.tar.gz
tar xzvf cilium-linux-amd64.tar.gz
sudo mv cilium /usr/local/bin
```

📁 2. Directory Structure

```
~/intent-routing/
├── policies/
```

```
│   └── latency-intent.json
├── scripts/
│   └── simulate-telemetry.py
├── src/
│   └── intent_router.c
├── test/
│   └── mock_services.sh
└── README.md
```

🛠️ 3. Tasks You Can Complete Now

3.1 🚦 Define Performance Intents

Create `policies/latency-intent.json`:

```
{
  "intent": "low-latency",
  "threshold_ms": 100,
  "action": "reroute"
}
```

3.2 🧠 Simulate Decision Engine

Create `scripts/simulate-telemetry.py`:

```
import json, random

with open("../policies/latency-intent.json") as f:
    intent = json.load(f)

latency = random.randint(50, 200)
if latency > intent["threshold_ms"]:
    print(f"Latency {latency}ms > {intent['threshold_ms']}ms →
{intent['action'].upper()} triggered")
else:
    print(f"Latency {latency}ms OK → No action")
```

Run it:

```
python3 scripts/simulate-telemetry.py
```

📡 4. Mock Network Conditions for Testing

4.1 Use `tc` to simulate latency:

```
sudo tc qdisc add dev lo root netem delay 150ms
ping 127.0.0.1
sudo tc qdisc del dev lo root netem
```

🔧 5. Start eBPF Program in C (intent_router.c)

In `src/intent_router.c`:

```c
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>

SEC("kprobe/tcp_v4_connect")
int bpf_prog(struct pt_regs *ctx) {
    bpf_printk("Intercepted TCP connection\n");
    return 0;
}

char LICENSE[] SEC("license") = "GPL";
```

🧪 Build & Load

```
clang -O2 -g -target bpf -c src/intent_router.c -o intent_router.o
sudo bpftool prog load intent_router.o /sys/fs/bpf/intent_router
sudo bpftool prog attach /sys/fs/bpf/intent_router kprobe/tcp_v4_connect
```

Check logs:

```
sudo cat /sys/kernel/debug/tracing/trace_pipe
```

🔗 6. Integrate with Cilium (Future Step)

Later, integrate with Cilium's `CNI` and `Hubble` to:

- Export metrics from eBPF
- Feed data to Grafana
- Combine with CiliumPolicy + L7 Intent

🧪 7. Optional: Mock Kubernetes Traffic

```
kubectl run svc-a --image=nginx
kubectl run svc-b --image=nginx
kubectl expose pod svc-b --port=80 --target-port=80
kubectl exec svc-a -- curl svc-b
```

📌 Summary

You can:

- Define and test intent policy parsing
- Simulate telemetry conditions
- Mock reroute logic
- Begin writing & testing eBPF hooks in C
- Prepare for integration with Component 1's data layer

> Need help writing Makefile, Cilium integration script, or converting this to a PDF? Just ask!